# Refresher Module | OS Monsoon'25

## 1. Introduction to Pointers

### 1.1 What is a Pointer?

> A pointer in C is a variable that stores the **memory address** of another variable.

It allows indirect access and manipulation of data in memory, a critical feature for operating systems.

```
int a = 10;
int *p = &a;
```

Here, `p` stores the address of variable `a`, i.e., `p` points to `a`.

### 1.2 Why are Pointers Important?

- **Memory Efficiency:** You can manipulate large data structures like arrays or structs by passing addresses instead of copying entire values.
- **Dynamic Memory:** Enables usage of `malloc`, `calloc`, `realloc`, and `free`.
- **Data Structures:** Pointers are the backbone of linked lists, trees, graphs.
- **Function Arguments:** Enables call by reference.
- **System Programming:** Used for manipulating hardware, I/O operations, and memory-mapped devices.
- **The Gate to Segmentation Faults:** Mishandled pointers are the most common source of crashes in C, making pointer literacy essential.

---

# 2. Referencing and Dereferencing

## 2.1 The `&` Operator → Referencing

`&` is the address-of operator. It gives the memory address of a variable.

```
int x = 42;
int *p = &x; // p now holds the address of x
```

## 2.2 The `*` Operator → Dereferencing

`*` is the dereference operator. It accesses the value stored at the memory address.

```
printf("%d\n", *p); // prints 42
```

## 2.3 Overall Picture

### Input

```
#include <stdio.h>

int main() {

    int x = 10;
```

```
    int *ip = &x;

    // Same value is printed
    printf("%d\n",x);
    printf("%d\n\n",*ip);

    // Same value is printed
    printf("%p\n",ip);
    printf("%p\n\n",&x);

    printf("%p\n",&ip);

    return 0;
}
```

### Output

```
10
10

0x7ffdb8b2dcec
0x7ffdb8b2dcec

0x7ffdb8b2dce0
```

| Operation | Pointer | Normal Variable |
|---|---|---|
| Obtain Value | Yes | Yes |
| Obtain address using & | Yes | Yes |
| Dereference using * | Yes | No |

# 3. Pointer Arithmetic

A general overview of pointer arithmetic

```
datatype *p = &a;
p + n = &a + n * sizeof(datatype);
```

Pointer arithmetic is scaled by the size of the data type →

```
int arr[] = {10, 20, 30};
int *p = arr;
p + 1 // points to arr[1], i.e., 4 bytes forward (if int is 4 bytes)
```

## 3.1 Increment and Decrement

```
p++;   // moves pointer to next element
p--;   // moves pointer to previous element
```

## 3.2 Pointer Difference

```
int diff = &arr[2] - &arr[0]; // yields 2 (not byte difference, but element coun
t)
```

## 3.3 Comparison

```
if (p1 < p2) // safe only if p1 and p2 point into the same array
```

## 3.4 Quiz Time

### Question →

```
#include <stdio.h>

int main() {

        unsigned int x = 3251;
        char *cp = &x;
        unsigned char c1 = *cp++, c2 = *cp;
        printf("%d\n",c1);
        printf("%d\n",c2);
```

```
    return 0;
}
```

# 4. Precedence Rule

## 4.1 Let's Understand

```
*p++     // increments pointer, then dereferences old location
*(p++)   // same as *p++, just clearer syntax
(*p)++   // increments value at pointer location
```

## 4.2 Example

```
int arr[] = {1, 2, 3};
int *p = arr;

printf("%d\n", *p++);    // 1, then p points to 2
printf("%d\n", (*p)++);  // 2, then arr[1] becomes 3
```

# 5. `void *` → The Chameleon of Pointers

## 5.1 Nature of `void *`

The `void *` pointer, often called the *generic pointer,* is a powerful feature in C. It's used when the type of data the pointer will point to is not known at the time of writing code. This enables **generic programming**, such as writing data structure libraries, memory allocators, and system-level interfaces.

Unlike other pointers, a `void *` doesn't have a data type associated with it. That means it doesn't know how many bytes it refers to, so →

- You **can assign** any type's address to a `void *`.
- You **cannot dereference** a `void *` without typecasting it first.
- You **cannot perform pointer arithmetic** on `void *` in standard C.

## 5.2 Why use `void *` ?

- **Memory allocation:** Functions like `malloc()` return a `void *`, which can be assigned to any pointer type.
- **Generic functions:** Standard library functions accept `void *` to operate on any type of data.
- **Function parameters:** When building polymorphic behavior using callbacks or event handlers.

## 5.3 Example: Assigning and Casting

```
void *p;
int a = 99;
p = &a;

// Compiler error: can't dereference void * directly
// printf("%d\n", *p); // ❌

// Correct: cast before dereferencing
printf("%d\n", *(int *)p); // ✔️
```

The `(int *)` cast tells the compiler to treat `p` as if it were pointing to an `int`, so it knows to dereference 4 bytes (on a 32-bit system).

## 5.4 Casting with Other Types

```
float pi = 3.14;
void *vp = &pi;
printf("%f\n", *(float *)vp);

char c = 'Z';
vp = &c;
printf("%c\n", *(char *)vp);
```

## 5.5 Using `void *` with Malloc

```
int *arr = (int *)malloc(5 * sizeof(int));
```

Here, `malloc()` returns a `void *`, and we cast it to `int *` to store integers.

## 5.6 Building a Generic Swap Function

```
void swap(void *a, void *b, size_t size) {
    char *pa = (char *)a;
    char *pb = (char *)b;
    for (size_t i = 0; i < size; i++) {
        char temp = pa[i];
        pa[i] = pb[i];
        pb[i] = temp;
    }
}
```

Usage:

```
int x = 10, y = 20;
swap(&x, &y, sizeof(int));

float f1 = 3.14, f2 = 2.71;
swap(&f1, &f2, sizeof(float));
```

This kind of code is foundational for writing standard library implementations.

## 5.7 What You Cannot Do with `void *`

```
void *p = malloc(sizeof(int));
*p = 10; // ❌ Error: compiler doesn't know the type
*(int *)p = 10; // ✔️ Correct: explicitly cast to int *

   p++; // ❌ Invalid: pointer arithmetic on void * is not allowed in standard
C
```

---

# 6. Function Pointers

## 6.1 What Is a Function Pointer?

You already know what a function is:

```
int add(int a, int b) {
   return a + b;
}
```

You call it like this:

```
int result = add(2, 3);
```

So what is a **function pointer**?

A function pointer is a **variable that stores the address of a function**.
Just like you can store the address of an `int` in an `int*`, you can store the address of a function in a function pointer.

## 6.2 Why Not Just Use Regular Functions?

### Choose Function at Runtime (Event Driven Programming)

You don’t always know which function to call until your program is running. Function pointers let you pick one **dynamically**, just like picking a variable.

### Reusability via Callbacks

You can pass function pointers into other functions as **arguments** to create reusable, generic behavior (like event handlers or comparators).

### Dispatch Tables

Arrays of function pointers can simulate dynamic dispatch or function selection, which is crucial in systems like OS kernels and interpreters.

### Extensibility

Plugins and drivers can register their behavior via function pointers without changing the core logic.

## 6.3 Syntax and Examples

### Step 1: Declare a Function

```
int add(int a, int b) {
    return a + b;
}
```

### Step 2: Declare a Function Pointer

```
int (*fptr)(int, int);
```

> This means: fptr is a pointer to a function taking two int arguments and returning an int.

### Step 3: Assign the Function to the Pointer

```
fptr = add; // or fptr = &add;
```

### Step 4: Call the Function Through the Pointer

```
int result = fptr(5, 3);  // works exactly like add(5, 3)
```

## 6.4 Use Case → Mini Calculator

Let's say you want to create a calculator with multiple operations →

```
#include <stdio.h>

int add(int a, int b) { return a + b; }
int sub(int a, int b) { return a - b; }
int mul(int a, int b) { return a * b; }

int (*ops[])(int, int) = {add, sub, mul};

int main() {
    int op = 2; // 0 for add, 1 for sub, 2 for mul
    printf("%d\n", ops[op](10, 3));
    return 0;
}
```

Now `ops[op]` dynamically calls whichever operation is selected, without `if-else` chains!

## 6.5 How OS Uses Function Pointers?

In an Operating System, **function pointers are critical** for writing flexible, modular, and extensible code.

### System Call Table

In Linux, every system call has a number. The kernel keeps a **syscall table,** an array of function pointers →

```
typedef int (*syscall_ptr_t)(void);
syscall_ptr_t syscall_table[256];

// When syscall #5 is triggered
int syscall_num = 5;
syscall_table[syscall_num]();
```

This allows the OS to **map syscall numbers to real functions** without writing a giant switch-case.

### Interrupt Handlers

Drivers register their **interrupt service routines** using function pointers →

```
typedef void (*irq_handler_t)(void);
irq_handler_t irq_vector[256];

void register_irq(int n, irq_handler_t handler) {
   irq_vector[n] = handler;
}
```

When the interrupt occurs, the registered handler is called:

```
irq_vector[15](); // call handler for interrupt 15
```

### The `__start` Function in Linking

In Linux and many Unix-like systems, the **actual entry point of a C program is not** `main()`, but a function called `__start`.

- It's the real entry point specified in the ELF header of theexecutable.
- It's usually written in **assembly** and is part of the system startup code (CRT: C Runtime).
- It prepares the stack, environment, and arguments and then calls `main()`.

```
_start:
   setup stack and registers
   call main
   call exit
```

Function pointers are used inside the **startup routines** to call into C code (like `main`) from low-level system  calls. This is one of the earliest places where **code and data become interchangeable**.

---

# 7. Double Pointers

Before jumping to 2D arrays, let's first understand how a dynamic array grows.

Simple Dynamic Array Example →

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *arr = NULL;
    int capacity = 2; // start small
    int size = 0;
    arr = malloc(capacity * sizeof(int));

    for (int i = 0; i < 10; i++) {
        if (size == capacity) {
            capacity *= 2; // double the size
            arr = realloc(arr, capacity * sizeof(int));
        }
        arr[size++] = i * 10; // insert element
    }

    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }

    free(arr);
    return 0;
}
```

This pattern is the backbone of `std::vector` in C++ and dynamic lists in other languages.

## 7.1 What is a Double Pointer?

A **double pointer** is a pointer that stores the address of another pointer. Think of it as a "pointer to pointer."

```c
int a = 10;
int *p = &a;
int **pp = &p;
```

```
printf("%d\n", **pp); // prints 10
```

Here's what's happening:

- `a` is an integer
- `p` is a pointer to `a`
- `pp` is a pointer to `p`
- `pp` gives `p`
- `*pp` gives the value of `a`

## 7.2 Why Use Double Pointers?

### Use Case 1: Dynamic 2D Arrays

```
int **matrix = malloc(rows * sizeof(int*));
for (int i = 0; i < rows; i++) {
    matrix[i] = malloc(cols * sizeof(int));
}
```

### Use Case 2: Modifying a Pointer in a Function

```
void allocate(int **ptr) {
    *ptr = malloc(sizeof(int));
    **ptr = 42;
}
```

### Use Case 3: Arrays of Strings

```
char *arr[] = {"hello", "world"};
char **p = arr;
printf("%s\n", *p);     // prints "hello"
```

## 7.3. Understanding `char*`, `char[]`, `char**`, and `void**`

## What is a String in C?

In C, a **string** is just an array of characters **terminated by a null character** (`'\0'`). This null character marks the end of the string. It's what differentiates →

- A character array: just a list of characters
- A C string: a null-terminated character array

```
char name[] = ['R','a','c','h','i','t']
char name1[] = "Rachit";  // char array with null terminator (7 bytes)
char *name2 = "Rachit";  // pointer to a string literal with '\0' at the end
```

If you forget to include `'\0'`, many string functions like `printf("%s", str)`, `strlen()`, `strcpy()` will keep reading memory until they accidentally hit a `'/0'`, which is undefined behavior.

## Visual Representation of name1 →

```
name1[] = ['R','a','c','h','i','t','\0']
```

## Differences Between `char[]` and `char*`

| Feature | `char[]` | `char*` |
|---|---|---|
| Memory Location | Allocated on stack | Points to literal or heap |
| Mutability | Can be modified | Literal version is read-only |
| Size | Includes `'\0'`, size fixed | Just pointer to first char |
| Terminator | Must end with `'\0'` | Must be `'\0'`-terminated manually |

## Example:

```
char str1[] = "hello"; // modifiable, lives on stack
char *str2 = "hello";  // read-only, lives in text segment
```

Trying to modify `str2[0] = 'H';` causes undefined behavior, but `str1[0] = 'H';` works fine.

## `char*` → A Pointer to a Character

This is used to represent strings in C →

```
char *str = "hello";
printf("%s\n", str);
```

`str` points to the first character of a null-terminated array of characters.

### `char**` → Pointer to a String Array

This can represent multiple strings, like `argv` in `main` →

```
int main(int argc, char **argv) {
    printf("Program name: %s\n", argv[0]);
}
```

Here, `argv` is a pointer to an array of `char*`, and `argv[0]` is the first string.

### `void**` → Generic Pointer to Pointer

Useful in APIs that modify data of unknown type →

```
void allocate(void **p, size_t size) {
    *p = malloc(size);
}

int *arr;
allocate((void**)&arr, 10 * sizeof(int));
```

This is the generic equivalent of `int**`, `float**`, etc.

#### Caution →

- Always match the level of indirection with the usage.
- Misuse leads to undefined behavior or segmentation faults.

---

# 8. Arrays

Arrays are stored in contiguous memory locations.

```
datatype name[size];    // Declaration
name = {el1, el2, el3, ... elk};    // Wrong Definition!

// Assignment in arrays is not allowed after declaration.

name[j] = elj;    // Piecewise assignment is allowed.
```

## 8.1 Declration of Arrays

```
datatype name[size] = {el1, el2, el3, ... elk};

// What is the difference between these declarations and definitions?

datatype name[] = {el1, el2, el3, ... elk};
```

### n-Dimensional Arrays

```
datatype name[s1][s2]...[sn];

// These are equivalent declarations.

datatype name[s1 * s2 * ... * sn];
```

## 8.2 Array Storage in Memory

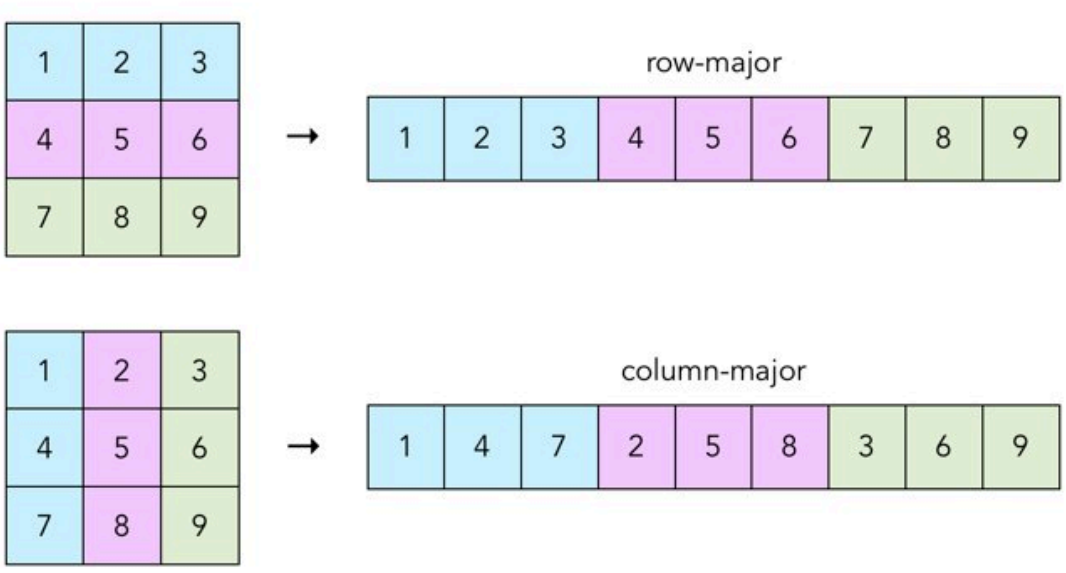
1. The elements of an array can be stored in a **column-major layout** or a **row-major layout**.
2. For an array stored in coulm-major layout, the elements of the columns are contiguous in memory.
3. In row-major layout, the elemts of the rows are contiguous. [This is what C uses]

### Row major →

```
A(i,j) element is at A[j + i * n_columns]
```

### Column major →

A(i,j) element is at A[i + j * n_rows]




### Why does it matter?

💡 OS Concept → **Caching**

https://craftofcoding.wordpress.com/2017/02/03/column-major-vs-row-major-arrays-does-it-matter/

## 8.3 Role of pointers in Array

```
int arr[10] = {1, 2, 3};
int *pa = arr;
```

```
*(pa + i) == arr[i] == *(arr + i)    // True

// in case i is out of bounds, i.e. in this case i > 10, arr[i] and *(arr + i) would
// throw error, but *(pa + i) would point to some garbage value.
```

1. Here space has been reserved for 10 integers at contiguous locations in the memory.
2. The name of the array points to the first byte of the address of the first array location.
3. For chars, referencing arr is the same as saying &arr[0].

C stores 2D arrays in **row-major** order.

## 8.4 Task → Reverse Array Using Pointers

```
void reverse(int *start, int *end) {
  while (start < end) {
    int temp = *start;
    *start = *end;
    *end = temp;
    start++;
    end--;
  }
}
```

## 8.5 Passing Arrays to Functions

### Which of the following is/are a valid function declaration?

1. `int func(int array[s1][s2][s3]);`
2. `int func(int array[][s2][s3]);`
3. `int func(int array[][][s3]);`
4. `int func(int array[][][]);`

### Answer

💡 1, 2

**Reason** → The number of columns should always be specified. (because C uses row-major storage).

---

# 9. Dynamic Memory Allocation

## 9.1 Malloc

The word “malloc” stands for memory allocation. We use malloc to dynamically allocate a single large block of memory with the specified size.

- malloc returns a pointer of type void which can be cast into a pointer of any form.
- malloc doesn’t initialize memory at execution time.
- If space is insufficient, allocation fails and returns a NULL pointer.

```
int *ptr = (int *)malloc(16 * sizeof(char));
```

Here ptr is typecast into an integer pointer. We have reserved 16 * 1 bytes worth of uninitialized memory.

💡 However the code given above is syntactically correct, but it doesn’t follow good coding practices. We must’nt use `sizeof(char)` while allocating space for integer pointers.

## 9.2 Calloc

The word “calloc” stands for contiguous memory allocation. We use malloc to dynamically allocate a single large block of memory with the specified size.

- calloc returns a pointer of type void which can be cast into a pointer of any form.
- calloc initializes each block with a default value ‘0’.

- If space is insufficient, allocation fails and returns a NULL pointer.

```
int *ptr = (int *)calloc(16, sizeof(int));
```

Here ptr is typecast into an integer pointer. We have reserved 16 contiguous segments in memory
each with 4 bytes worth of memory initialized to 0.

## 9.3 Realloc

The word “realloc” stands for re-allocation of previously allocated memory. We use realloc primarily when the original malloc or calloc was too small.

- realloc returns a pointer of type void which can be cast into a pointer of any form.
- realloc does not initialize each block.
- If space is insufficient, allocation fails and returns a NULL pointer.

```
int *ptr = (int *)malloc(16 * sizeof(int));
ptr = realloc(ptr, 32 * sizeof(int));
```

Here ptr is typecast into an integer pointer. We have reserved 164 bytes worth of memory initially. We are then reallocating 324 bytes worth of memory and pointing it to ptr.

## 9.4 Free

The free command is used to de-allocation of previously allocated memory.

```
int *ptr = (int *)malloc(16 * sizeof(int));
free(ptr);
```

Here ptr is typecast into an integer pointer. We have reserved 16*4 bytes worth of memory initially. We are then freeing the memory.

---

# 10. Stages of Compilation

How do you go from a .c file to the a.out executable?


1. **Preprocessing** ( `-E` flag)

   Preprocessor directives # are interpreted and commands are stripped out.

   - input → `.c` file
   - output → `.i` file

2. **Compilation** ( `-S` flag)

   Preprocessed code is translated to assembly instructions specific to the target
   processor architecture.

   - input → `.c` file
   - output → `.i` file

3. **Assembly** ( `-c` flag; view file using hexdump)

   Translate the assembly instructions to object code (actual instructions).

   - input → `.c` file
   - output → `.i` file

4. **Linking** ( `-o` flag; view file using hexdump)

   The object code generated has missing pieces of instructions. These are filled
   in during linking.

   - input → `.c` file
   - output → `.i` file

## 10.1 Preprocessing

The file with the result of preprocessing usually has "`.i`" suffix
The preprocessor carries out the following operations →

1. Inclusion of header files ( `'#include'` )

2. Macros substitution ( `'#define'` )
3. Conditional compilation ( `'#if'` , `'#ifdef'` , `'#else'` , `'#elif'` , `'#endif'` ).
4. The result of preprocessing is called a translation unit

## 10.2 Compilation

The file with the result of compilation usually has " `.s` " suffix

- .s files are source code files written in assembly code.
- The files contain assembly instructions to the processor in sequential order and are
  typically compiled based on a selected architecture.
- Machine dependent instructions (I/O) and early initialization such as setting up
  cache and memory.

```
00000000        push    ebp
00000001        mov     ebp, esp
00000003        movzx   ecx, [ebp+arg_0]
00000007        pop     ebp
00000008        movzx   dx, cl
0000000C        lea     eax, [edx+edx]
0000000F        add     eax, edx
00000011        shl     eax, 2
00000014        add     eax, edx
00000016        shr     eax, 8
00000019        sub     cl, al
0000001B        shr     cl, 1
0000001D        add     al, cl
0000001F        shr     al, 5
00000022        movzx   eax, al
00000025        retn
```

## 10.3 Assembly

The file with the result of assembly usually has " `.o` " suffix

- `.o` files are relocatable object codes..
  - Relocatable code is software whose execution address can be changed. A
    relocatable program might run at address 0 in one instance, and at 10000 in
    another.

- Translation of the .s file to its corresponding binary machine code equivalent.

## 10.4 Linking

The link editor creates an executable file (`a.out` file) from one or more `.o` files.

```
gcc -o simple simple.o
```

This command creates an executable file from simple.o and some standard libraries that gcc automatically links in.

---